VNR VJIET Name of the Experiment:

Name of Laboratory: _____

_____ Date: _____

WEEK – 5

Feature Construction: (Use packages that are applicable)

1. Dummy coding categorical(nominal) variables.
2. Encoding categorical(ordinal) variables.
3. Transforming numeric(continuous)features to categorical features

```
# Importing all the required packages
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, OrdinalEnco
der
```

```
df = pd.read_csv('homeprices.csv')
df
```

```
               town  area   price
0   monroe township  2600  550000
1   monroe township  3000  565000
2   monroe township  3200  610000
3   monroe township  3600  680000
4   monroe township  4000  725000
5      west windsor  2600  585000
6      west windsor  2800  615000
7      west windsor  3300  650000
8      west windsor  3600  710000
9       robinsville  2600  575000
10      robinsville  2900  600000
11      robinsville  3100  620000
12      robinsville  3600  695000
```

1. Dummy coding categorical(nominal) variables.

- Nominal data is categorical data that is qualitative, meaning that it does not have any inherent order. E.g Customer satisfaction ratings (e.g., 1-5 stars) ,Educational level (e.g., high school diploma, bachelor's degree, master's degree) ,Military rank (e.g., private, corporal, sergeant)

```
# As town is categorical
dummies = pd.get_dummies(df.town)
dummies
```

```
    monroe township  robinsville  west windsor
0                 1            0             0
```

```
1                 1            0             0
2                 1            0             0
3                 1            0             0
4                 1            0             0
5                 0            0             1
6                 0            0             1
7                 0            0             1
8                 0            0             1
9                 0            1             0
10                0            1             0
11                0            1             0
12                0            1             0
```

```
# avoid dummy variable trap, by dropping one column
dummies = dummies.drop(['robinsville'],axis=1)
dummies
```

```
    monroe township  west windsor
0                 1             0
1                 1             0
2                 1             0
3                 1             0
4                 1             0
5                 0             1
6                 0             1
7                 0             1
8                 0             1
9                 0             0
10                0             0
11                0             0
12                0             0
```

```
# Add dummies to the original df
merged= pd.concat([df, dummies], axis=1)
merged
```

```
               town  area   price  monroe township  west windsor
0   monroe township  2600  550000                1             0
1   monroe township  3000  565000                1             0
2   monroe township  3200  610000                1             0
3   monroe township  3600  680000                1             0
4   monroe township  4000  725000                1             0
5      west windsor  2600  585000                0             1
6      west windsor  2800  615000                0             1
7      west windsor  3300  650000                0             1
8      west windsor  3600  710000                0             1
9       robinsville  2600  575000                0             0
10      robinsville  2900  600000                0             0
11      robinsville  3100  620000                0             0
12      robinsville  3600  695000                0             0
```

```
# drop town
final = merged.drop(['town'], axis=1)
final
```

```
    area   price  monroe township  west windsor
0   2600  550000                1             0
```

```
1   3000  565000                1              0
2   3200  610000                1              0
3   3600  680000                1              0
4   4000  725000                1              0
5   2600  585000                0              1
6   2800  615000                0              1
7   3300  650000                0              1
8   3600  710000                0              1
9   2600  575000                0              0
10  2900  600000                0              0
11  3100  620000                0              0
12  3600  695000                0              0
```

```
# Create a OneHotEncoder object
x.toarray() = OneHotEncoder(handle_unknown='ignore') # the encoder will si
mply ignore the unknown categories and will not create any new features fo
r them.

# Fit the encoder to the data, Transform the data
df_encoded = encoder.fit_transform(df[['town']])

# Print the one-hot encoded data
pd.DataFrame(df_encoded.toarray())
```

```
      0    1    2
0   1.0  0.0  0.0
1   1.0  0.0  0.0
2   1.0  0.0  0.0
3   1.0  0.0  0.0
4   1.0  0.0  0.0
5   0.0  0.0  1.0
6   0.0  0.0  1.0
7   0.0  0.0  1.0
8   0.0  0.0  1.0
9   0.0  1.0  0.0
10  0.0  1.0  0.0
11  0.0  1.0  0.0
12  0.0  1.0  0.0
```

Here are some of the benefits of using one-hot encoding:

1. It makes it easy to use categorical variables in machine learning models.
2. It preserves the information in the categorical variables.
3. It is easy to understand and implement.

However, there are also some limitations to using one-hot encoding:

1. It can increase the dimensionality of the data.
2. It can be sensitive to outliers.
3. It can be difficult to interpret the results of machine learning models that use one-hot encoded variables.

When to use one-hot encoding vs dummy encoding:

- One-hot encoding is generally preferred over dummy encoding, as it is more efficient and less prone to overfitting.
- Dummy encoding should be used when the number of categories in the categorical variable is small and the categories have a natural order.

2. Encoding categorical(ordinal) variables.

- Ordinal data is categorical data that is quantitative, meaning that it has a natural order. E.g. Eye color, Hair color ,Nationality ,Religion ,Political affiliation

Label Encoder

A label encoder is a machine learning technique used to convert categorical variables into numerical variables. It is a simple and effective way to prepare data for machine learning models, as most machine learning models only accept numerical data.

```
iris = pd.read_csv('iris.csv')
iris
```

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   ...            ...           ...            ...           ...
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            Species
0       Iris-setosa
1       Iris-setosa
2       Iris-setosa
3       Iris-setosa
4       Iris-setosa
..              ...
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

[150 rows x 6 columns]
```

```
encoder = LabelEncoder()
iris_encoded = encoder.fit_transform(iris['Species'])
print(iris_encoded)
iris.Species = iris_encoded
iris
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]

      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Specie
s
0      1            5.1           3.5            1.4           0.2
0
```

```
1      2            4.9           3.0           1.4           0.2
0
2      3            4.7           3.2           1.3           0.2
0
3      4            4.6           3.1           1.5           0.2
0
4      5            5.0           3.6           1.4           0.2
0
..   ...            ...           ...           ...           ...      ..
.
145  146            6.7           3.0           5.2           2.3
2
146  147            6.3           2.5           5.0           1.9
2
147  148            6.5           3.0           5.2           2.0
2
148  149            6.2           3.4           5.4           2.3
2
149  150            5.9           3.0           5.1           1.8
2

[150 rows x 6 columns]
```

Observation:

- Species attribute is the ordinal attribute, so it needs to be converted into numeric
- 0 -> Iris-setosa , 1 -> Iris-versicolor, 2 -> Iris-virginica

Benefits of using label encoders:

- They are simple to understand and implement.
- They are efficient and scalable.
- They preserve the information in the categorical variable.

Limitations of using label encoders:

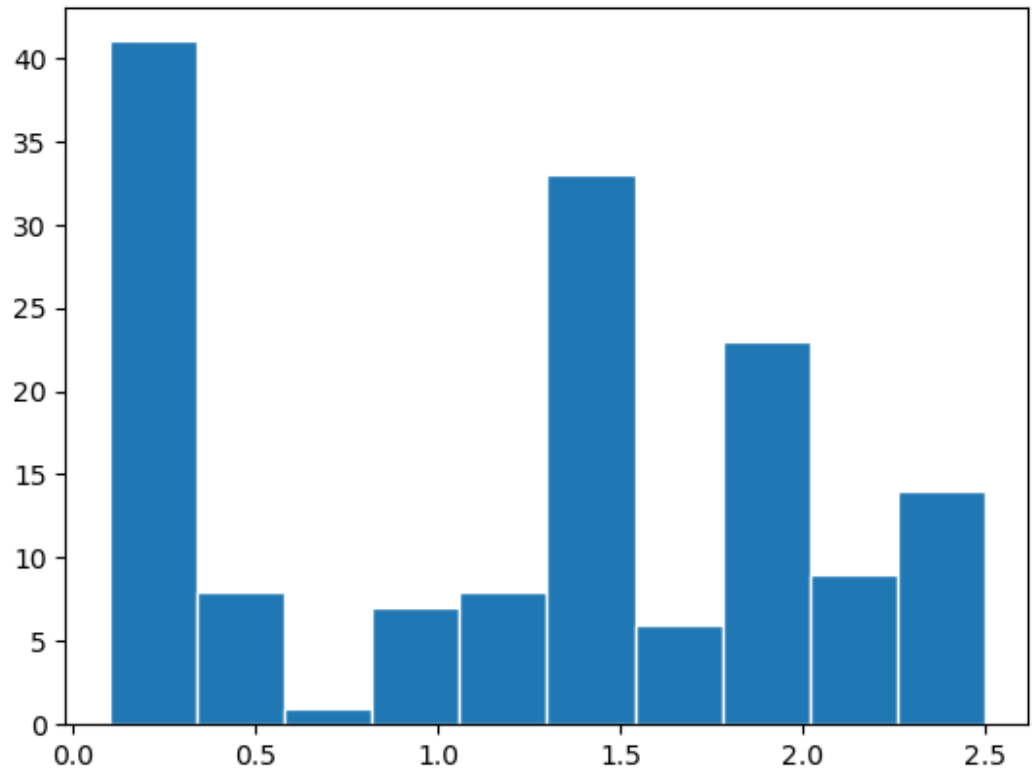
- They can introduce an artificial order to the categorical variables.
- They can be sensitive to outliers.
- They can be difficult to interpret the results of machine learning models that use label encoded variables.

3. Transforming numeric(continuous)features to categorical features

Binning:

- This involves dividing the numeric feature into a number of bins, and then assigning each value in the feature to a bin. The bins can be created using equal intervals, or by using a more sophisticated method such as quantile binning

```
plt.hist(iris.PetalWidthCm, edgecolor='w')
plt.show()
```




```
iris['petal_category'] = pd.cut(iris.PetalWidthCm, bins=[0, 1, 2, 2.5], la
bels=['small', 'medium', 'large'])
print(iris['petal_category'])
```

```
0       small
1       small
2       small
3       small
4       small
        ...
145     large
146    medium
147    medium
148     large
149    medium
Name: petal_category, Length: 150, dtype: category
Categories (3, object): ['small' < 'medium' < 'large']
```

```
iris['petal_category'].value_counts()
```

```
medium    70
small     57
large     23
Name: petal_category, dtype: int64
```

Ordinal Encoder

- An ordinal encoder is a machine learning technique used to convert ordinal categorical variables into numerical variables. Ordinal categorical variables are categorical variables that have a natural order, such as customer satisfaction ratings (e.g., 1-5 stars) or educational level (e.g., high school diploma, bachelor's degree, master's degree).

```
encoder = OrdinalEncoder()
```

```
iris['petal_category'] = encoder.fit_transform(iris['petal_category'])
iris
```

```
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Specie
s  \
0      1            5.1           3.5            1.4           0.2
0
1      2            4.9           3.0            1.4           0.2
0
2      3            4.7           3.2            1.3           0.2
0
3      4            4.6           3.1            1.5           0.2
0
4      5            5.0           3.6            1.4           0.2
0
..   ...            ...           ...            ...           ...      ..
.
145  146            6.7           3.0            5.2           2.3
2
146  147            6.3           2.5            5.0           1.9
2
147  148            6.5           3.0            5.2           2.0
2
148  149            6.2           3.4            5.4           2.3
2
149  150            5.9           3.0            5.1           1.8
2

     petal_category
0                 2
1                 2
2                 2
3                 2
4                 2
..              ...
145               0
146               1
147               1
148               0
149               1

[150 rows x 7 columns]
```

Observation:

- The attribute PetalLengthCm is converted into Ordinal type and then it is encoded into continuous data
- Scikitlearn Ordinal Encoder is used to transform categorical data into continuous data

Benefits of using ordinal encoders:

- They are simple to understand and implement.
- They are efficient and scalable.
- They preserve the information in the ordinal categorical variable.
- They can be used with machine learning models that expect numerical features.

Limitations of using ordinal encoders:

- They can be sensitive to outliers.
- It can be difficult to interpret the results of machine learning models that use ordinal encoded variables.